

Localización de emisores individuales con haces dona: qué se hizo, qué se resolvió y cómo seguir hacia p-MINFLUX

flor-choque (FlorJDC) · github.com/FlorJDC/donut-beam-localization · 28/09/2026 · detalle completo en [paper/main.pdf](#) (13 pp.)

Resumen. Construimos un estudio de simulación reproducible de la localización con haces dona (MINFLUX) y lo usamos para responder qué limita la precisión y qué introduce sesgo. Del trabajo sale un paquete Python propio, un script por figura, 254 tests, 217 números con procedencia y un manuscrito de 13 páginas; cada resultado lo reprodujo un verificador independiente. Después lo contrastamos con el simulador SimuFLUX (Marin & Ries, 2026) y lo llevamos al caso de interés: **p-MINFLUX no iterativo**, con láser de 20 MHz, 4 pulsos entrelazados de igual duración y L fijo. El resultado central es que un estimador de máxima verosimilitud (MLE) con el modelo correcto alcanza el límite de Cramér–Rao (CRB). Casi todos los sesgos aparecen cuando el modelo omite algo: el fondo, la desalineación del patrón o el cruce entre ventanas temporales.

1. Qué se hizo y cómo

Modelo. Un emisor en r se expone sucesivamente a $K = 4$ haces con un mínimo de intensidad. Usamos el patrón triangular (TCP): 3 donas en un círculo de diámetro L más una en el centro. Los fotones de cada exposición siguen una multinomial con $p_i \propto I_i(r) + \text{fondo}$ (Balzarotti et al. 2017). La dona es Laguerre–Gauss (LG) con $\text{fwhm} = 300$ nm, contrastada con una dona vectorial de alta apertura (Richards–Wolf, $\text{NA} = 1.4$). La precisión se mide con el CRB, $\sigma = \sqrt{((\Sigma_{xx} + \Sigma_{yy})/2)}$, y con Monte Carlo de estimadores. Unidades: nm.

Método de trabajo. El trabajo se organizó en rondas acotadas de agentes (metodología *agent-team*). Cada ronda tiene un planificador, workers que programan con tests y un verificador que reproduce cada número por un camino propio sin ver el razonamiento del worker. Un resultado refutado no entra al texto: se corrige o se declara abierto. "Terminado" lo define un test de aceptación escrito antes de empezar y protegido por hash. Hubo 5 rondas para el paquete y el manuscrito, y 1 ronda para este informe. Todo queda registrado en [equipo/](#).

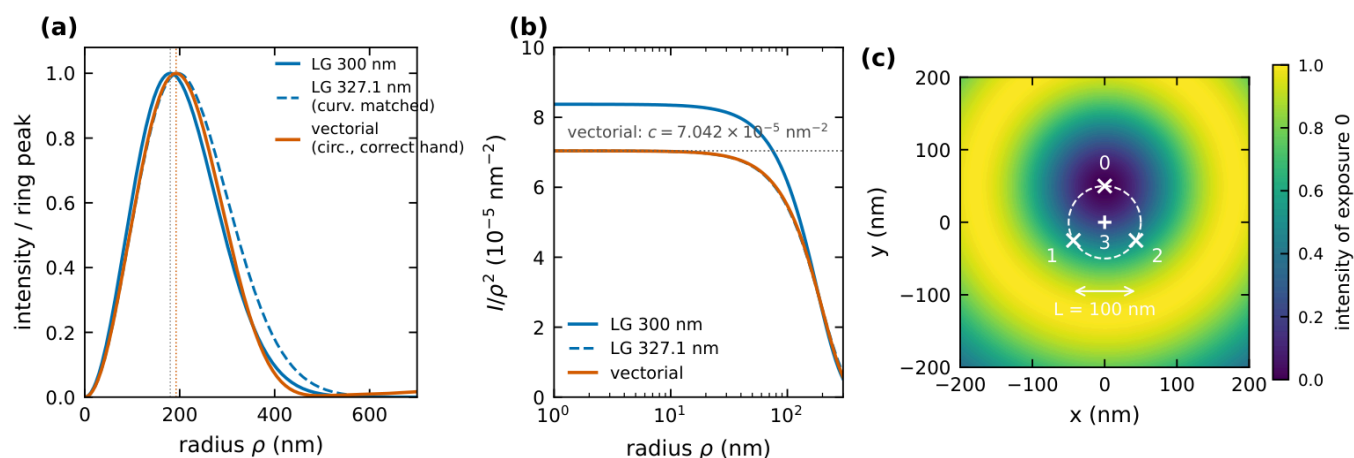


Figura 1. Modelo del trabajo. Perfil radial de la dona LG ($\text{fwhm} = 300$ nm), de la LG igualada por curvatura y de la dona vectorial (a). Curvatura cerca del cero, I/ρ^2 (b). Patrón TCP: 3 donas en un círculo de diámetro $L = 100$ nm más una en el centro; el mapa muestra la intensidad de una exposición (c). [scripts/fig_1_schematic.py](#) [src:fig1]

2. Problemas, cómo se encararon y qué se obtuvo

2.1 ¿Qué modelo de dona usar?

Cómo Implementamos la dona vectorial (máscara de vórtice y polarización circular) y la comparamos con la LG, en la forma cerca del cero y en el CRB resultante.

Resultado Una LG con la misma curvatura en el cero reproduce el CRB vectorial (diferencia < 0.4 %), así que el modelo LG alcanza. Si la polarización tiene la handedness opuesta, el cero se llena hasta el 85 % del pico y el CRB crece unas 90 veces. [src:vectorial]

Queda Aberraciones reales (origen físico de un cero imperfecto). El diámetro de la dona frente a PyFocus con filling factor 1 no coincide (428 frente a 401 nm).

2.2 ¿Cuál es la precisión límite, y qué pasa en el centro del patrón?

Cómo Calculamos la información de Fisher numéricamente en todo el campo y derivamos formas cerradas para el centro.

Resultado Sin fondo, el CRB es discontinuo en el centro. El límite $r \rightarrow 0$ vale 1.61 y el valor puntual de Balzarotti (Eq. S27), 1.80 ($L = 50$, $N = 100$). La razón es 0.890, cercana a $2/\sqrt{5}$, que es el valor exacto para un cero cuadrático. Con fondo la discontinuidad desaparece. El CRB escala como $\sigma \propto L^{1.00} N^{-0.50}$ para $L \ll \text{fwhm}$. Bastan 13 fotones para 5 nm, cuando una cámara ideal necesita 400. [src:crb_center] [src:scaling]

2.3 ¿Qué estimador alcanza ese límite?

Cómo Monte Carlo del MLE frente a los estimadores lineales LMS y mLMS, dentro y fuera del TCP.

Resultado Con fondo, el MLE es eficiente: eficiencia 0.991 con SBR = 10. LMS y mLMS se sesgan lejos del centro: -14.4 y -5.2 nm a 25 nm, contra 0.7 del MLE. Sin fondo el modelo no es regular en el centro, y el MLE queda por debajo del límite (0.84 del CRB) pero sesgado hacia el centro. [src:estimators]

Queda Confirmar la convención SBR = 10 que se usa para medir la eficiencia.

2.4 El cero real no es perfecto: ¿conviene achicar L ?

Cómo Barrido del CRB en función de L para distintos ceros residuales $\varepsilon = I(0)/I_{\text{max}}$.

Resultado No siempre: hay un L óptimo $\approx 0.78 \cdot \text{fwhm} \cdot \sqrt{\varepsilon}$ para $\varepsilon \lesssim 0.01$. Vale 10.5, 23 y 50 nm para $\varepsilon = 0.002$, 0.01 y 0.05. Es el criterio para elegir el L fijo. [src:zero_depth]

Queda Optimizar L con ε y fondo juntos, usando valores medidos.

2.5 Las donas no están donde se cree

Cómo Perturbamos al azar las posiciones de las donas (δ) y estimamos con el patrón nominal y con el verdadero.

Resultado Con el patrón nominal, el sesgo en el centro es $\approx 0.78 \cdot \delta$: 7.7 nm con $\delta = 10$. Con el patrón verdadero baja a 0.18 nm, así que la solución es calibrar. Para un sesgo menor que 1 nm hace falta $\delta \lesssim 1.3$ nm (1/0.78). [src:misalignment]

Queda Simular cómo se mide el patrón y con qué error.

2.6 Fondo mal modelado nuevo

Cómo SimuFLUX muestra que ignorar el fondo sesga fuertemente el estimador (su Fig. 2d). Hicimos el análogo para TCP fijo con MLE: fondo igual en las 4 ventanas, $N = 500$, y cuatro variantes del estimador (sin fondo, fondo exacto, fondo libre, fondo con error de $\pm 30\%$). Código en `background.py`.

Resultado Si el MLE ignora el fondo, lee la señal extra de la exposición central como un emisor más alejado y sesga la posición hacia afuera: $\approx +5$ nm en x y $+3$ nm en y con SBR = 5, y $\approx +2$ nm con SBR = 20. Ajustar el fondo como tercer parámetro elimina el sesgo y cuesta poco: el CRB sube 0–4 % hasta 20 nm del centro y 20–26 % en $r = L/2$. Un error de $\pm 30\%$ en el fondo deja ≤ 1.6 nm (Fig. 3). [src:background]

Queda Fondo distinto en cada ventana y cómo medirlo en los datos.

2.7 Parpadeo del fluoróforo nuevo

Cómo SimuFLUX muestra que en MINFLUX secuencial el parpadeo suma un error σ_{fl} que no depende del número de fotones. Simulamos un parpadeo on/off de 100 μs durante 400 μs , con exposiciones en bloques (secuencial) o alternadas cada 12.5 ns (p-MINFLUX). Código en `pminflux.py`.

Resultado En el modo secuencial, σ_{fl} va de 16 nm (1 pasada del patrón) a 2.2 nm (25 pasadas), y se cumple $\text{STD}^2 = \sigma_{\text{fl}}^2 + \sigma^2$, como en SimuFLUX. Con el entrelazado, las 4 exposiciones ven el mismo tiempo "on" y $\sigma_{\text{fl}} \lesssim 0.03$ nm. Es una ventaja concreta de p-MINFLUX (Fig. 2a). [src:pminflux_flicker]

Queda Parpadeo rápido (de ns a μs) y fotoblanqueo: el entrelazado protege solo si el parpadeo es mucho más lento que 50 ns.

2.8 Cruce entre ventanas por el tiempo de vida nuevo

Cómo Este efecto es propio de la excitación pulsada y SimuFLUX no lo trata. Un fotón excitado por un pulso llega con retardo exponencial de tiempo de vida τ y puede caer en la ventana siguiente. Derivamos la matriz de mezcla $p^{\text{obs}} = M p$, con $M_{ij} = (1-q)q^{(s_i-s_j) \bmod 4}/(1-q^4)$ y $q = e^{-T/\tau}$, y la validamos con Monte Carlo de fotones.

Resultado Con $\tau = 5$ ns, un 7.5 % de los fotones cae en la ventana siguiente. Si el estimador ignora M , el sesgo es de 3.4 nm ($\tau = 3$ ns) y de 8–13 nm ($\tau = 5$ ns, según el orden de los pulsos). Si incluye M , el sesgo desaparece y el CRB sube 4–19 % ($\tau = 3$ –5 ns) (Fig. 2b–c). M depende solo de τ y T , así que corregirlo no cuesta nada. [src:pminflux_crosstalk]

Queda Todo lo anterior supone ventanas alineadas con el pulso y sin respuesta del instrumento (IRF). Falta obtener M del histograma TCSPC real.

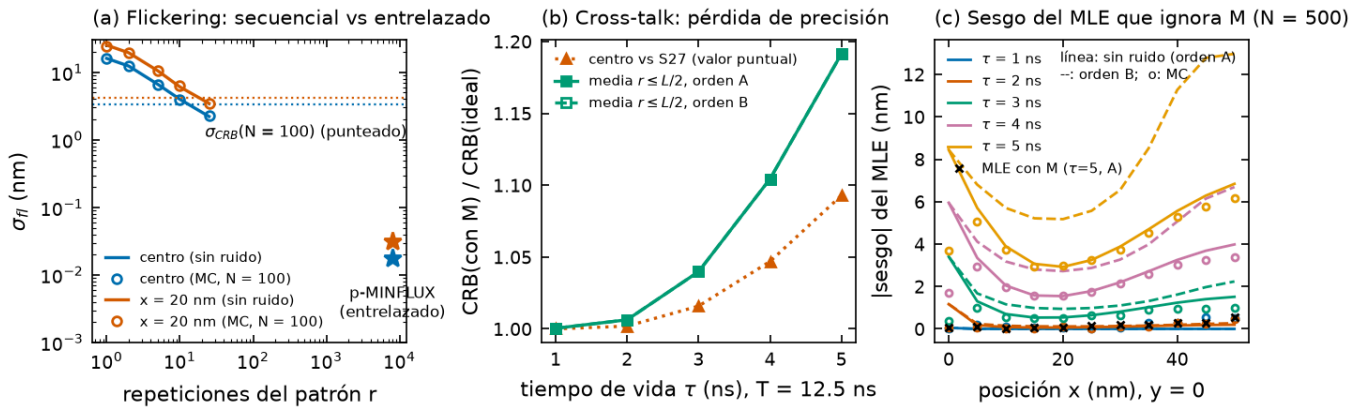


Figura 2. Temporización de p-MINFLUX ($L = 100$, fwhm = 300, $T = 12.5$ ns). Error por parpadeo: MINFLUX secuencial frente a entrelazado (estrellas) (a). Pérdida de precisión por el cruce entre ventanas con M en el modelo (b). Sesgo del MLE que ignora M ($N = 500$); las $\times$ corresponden al MLE con M (c). [scripts/fig_9_pminflux_timing.py](#) [src:fig9]

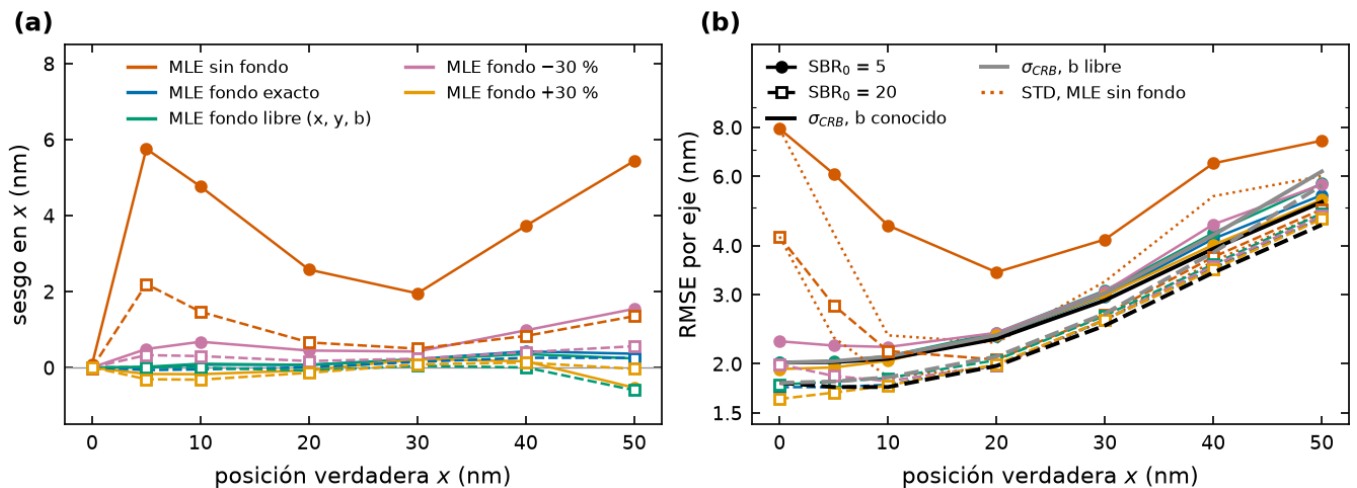


Figura 3. Fondo igual en las 4 ventanas con TCP fijo ($L = 100$, $N = 500$; SBR en el centro = 5, trazo lleno, y 20, discontinuo). Sesgo en x del MLE según cómo trata el fondo (a). RMSE frente a σ_{CRB} (b). [scripts/fig_10_background_bias.py](#) [src:fig10]

2.9 Problemas del propio proceso

- Cómo** Cada número del manuscrito es una macro que resuelve contra `data/paper_numbers.json`, y un script (`check_provenance.py`) rechaza cualquier número sin procedencia. Las figuras y los JSON se regeneran byte a byte.
- Resultado** Los verificadores refutaron afirmaciones en varias rondas, y se corrigieron antes de entrar al texto. En este informe, por ejemplo, el rango de σ/CRB con fondo exacto (0.97–1.06) y un artefacto de malla en el cruce entre ventanas para $\tau = 1$ ns. Un test fallaba con numpy 2 y se arregló. MINFLUX iterativo se estudió (0.51 nm con 1000 fotones, frente a 3.16 nm de una cámara ideal), pero no aplica a p-MINFLUX con L fijo.
- Queda** Los estudios nuevos (§2.6–2.8) todavía no están en el manuscrito ni en `reproduce.py`. Los tests nuevos no se corrieron en el entorno de referencia (Windows, Python 3.8).

3. Qué falta y cómo proseguir

Cada tarea reúne los "Queda" de la sección 2, en orden de prioridad. Quedan fuera de alcance, por el experimento: MINFLUX iterativo, dwell central distinto y tracking.

#	Tarea	Cómo	Terminado cuando
Prioridad 1 — un estimador p-MINFLUX completo, necesario para analizar datos sin sesgo			
1	Modelo completo (§2.4–2.8)	Una sola función de probabilidades en <code>pminflux.py</code> con centros medidos, ϵ , fondo por ventana y $M(\tau, T)$.	MLE insesgado ($< 0.1\sigma$) y $\sigma/CRB = 1 \pm 5\%$ en $r \leq L/2$, para $\tau = 1\text{--}5$ ns y SBR = 5–20.
2	M real (§2.8)	Calcular M desde el histograma TCSPC medido: IRF, retardo, ancho de ventana y canales descartados.	Con IRF = 0 reproduce la forma cerrada; sesgo por ± 0.5 ns de corrimiento cuantificado.
3	Calibración y L (§2.4–2.5)	Simular la medición de los centros y de ϵ ; optimizar L con ϵ y fondo juntos (<code>experiments.optimal_L</code>).	Tabla $L_{opt}(\epsilon, \text{SBR})$ y tolerancia de calibración.
4	Fondo en los datos (§2.6)	Decidir entre medir el fondo o ajustarlo libre; fondo distinto por ventana.	Error de fondo que se tolera para sesgo < 1 nm.

5	Prueba de punta a punta	Generar datos crudos simulados (tiempos de llegada con TCSPC, N de Poisson) y analizarlos con el estimador de la tarea 1.	Precisión $\approx$ CRB y sesgo < 1 nm.
Prioridad 2 — efectos que pueden cambiar los números			
6	Parpadeo rápido y blanqueo (§2.7)	Estados oscuros de ns a μ s y fotoblanqueo durante la medición.	σ_{fl} en función del tiempo de parpadeo.
7	Aberraciones (§2.1)	Fases de Zernike y máscara de vórtice desalineada en <code>vectorial.py</code> .	ϵ y sesgo en función de la amplitud de la aberración.
8	Deriva, vibración, dipolo, 3D	Emisor en movimiento, excitación según la orientación, emisor fuera del foco.	Error extra frente a la amplitud o el ángulo.
Prioridad 3 — consolidar el repositorio			
9	Integrar §2.6–2.8	Llevar las figuras 9–10 a <code>make_all_figures.py</code> , <code>compute_paper_numbers.py</code> , <code>reproduce.py</code> y <code>paper/main.tex</code> .	<code>check_provenance.py</code> y el test de aceptación pasan.
10	Validación cruzada (§2.1, §2.6)	Comparar con SimuFLUX (Python, código abierto) en un TCP estático con fondo, y con PyFocus para el diámetro de la dona.	Acuerdo, o diferencias explicadas.
11	Pendientes (§2.3, §2.9)	Confirmar SBR = 10 y la normalización del RMSE frente a Masullo et al. 2022; correr los tests en Python 3.8.	Puntos cerrados en <code>open_points.tex</code> .

Siguiente paso sugerido. Las tareas 1 y 2 juntas, como un trabajo de equipo `feature`. El criterio de la tarea 1 sirve como test de aceptación, y el resto del plan depende de tener ese estimador.

4. Reproducir y referencias

`python -m unittest discover -s tests` (≈ 30 s). Las figuras nuevas se regeneran con `python scripts/fig_9_pminflux_timing.py` y `python scripts/fig_10_background_bias.py` (≈ 1 min cada una, semilla 42), y el manuscrito con `python scripts/reproduce.py` (≈ 25 min). Este PDF se genera con `node informe/build_pdf.js`. Las etiquetas `[src:...]` resuelven en `equipo/2026-09-28_sintesis-pdf/out/provenance.json`.

1. F. Balzarotti et al., *Science* 355, 606 (2017) — MINFLUX.
2. L. A. Masullo et al., *Nano Lett.* 21, 840 (2021) — Pulsed Interleaved MINFLUX; y *Biophys. Rep.* 2, 100036 (2022).
3. Z. Marin, J. Ries, *Nat. Commun.* 17, 246 (2026) — SimuFLUX; nota en `docs/literature/D_simuflux.md`.